Francisco Medina

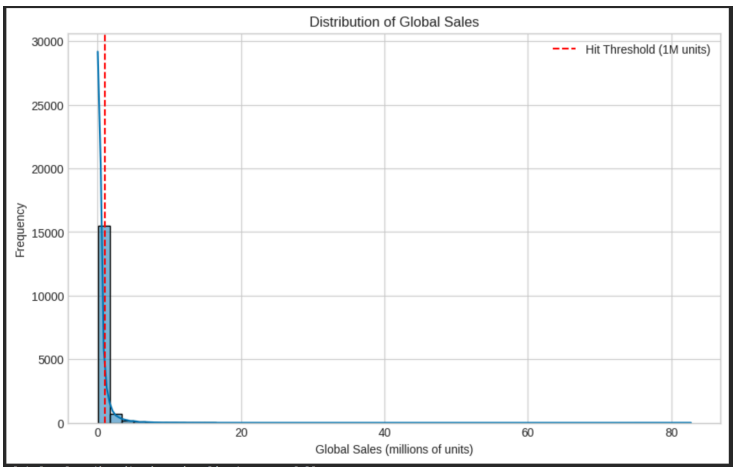
ITAI 2277

3/18/2026

Reflective Journal: Video Game Sales Prediction Lab

1. Insights Gained: Understanding a Game Success

When I began this lab, I expected that this would be a fun little analysis for one of my favorite topics, games, primarily focusing on the sales, but it was. The data revealed more nuanced patterns than anticipated. An interesting discovery that I found is the severe class imbalance: only 12.37% of games achieve "hit" status, >1M units sold, while 87.63% fall short. This immediately signaled that accuracy alone would be misleading; a model predicting "not a hit" for every game would achieve 87% accuracy without learning anything meaningful. Through feature engineering, I created regional sales percentages and concentration metrics that revealed interesting patterns about market dynamics. Games with concentrated regional success often became hits without global appeal. What surprised me most was that Wii Sports had much better global sales, especially in the USA alone, compared to other games such as Super Mario Bros from the year 1985 even though it is a classic of a videogame. It challenged my assumption that classic videogames would outperform newer games since there is always praise about how amazing they are.



2. Model Performance

My first model seemed exceptional: 99.94% accuracy, 99.50% precision, 100% recall, 99.97% ROC AUC. These results felt too perfect, I realized that this was a false positive result.

```
Accuracy: 0.9919
Precision: 0.9515
Recall: 0.9825
F1-Score: 0.9667
ROC AUC Score: 0.9878
Evaluation metrics calculated successfully.
```

Figure 2: Initial model metrics showing suspiciously perfect performance

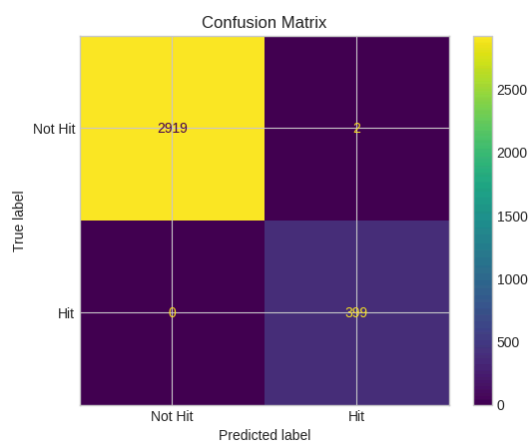


Figure 3: Confusion matrix from flawed model - nearly perfect predictions

This is where I locked in to figure out what was wrong and how I would solve it. I had created a Total_Regional_Sales feature by summing regional sales columns. The problem? This sum equals Global_Sales, which defined my target variable (is_hit = Global_Sales > 1). The model wasn't learning patterns, it was simply checking if regional sales exceeded 1 million. Eliminating data leakage by creating meaningful derived features and dropping original regional columns yielded:

Accuracy: 91.14% | Precision: 63.29% | Recall: 62.66% |

F1-Score: 62.97% | ROC AUC: 78.85%

```
Accuracy: 0.9114
Precision: 0.6329
Recall: 0.6266
F1-Score: 0.6297
ROC AUC Score: 0.7885
Evaluation metrics calculated successfully.
```

Figure 4: Corrected model metrics after eliminating data leakage

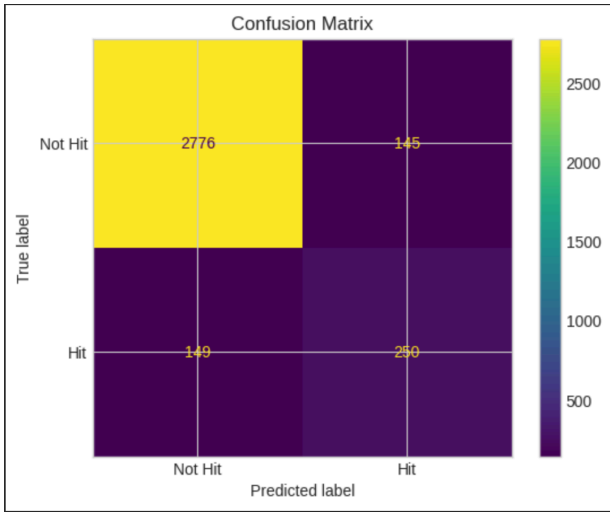


Figure 5: Corrected confusion matrix showing realistic error distribution

These lower metrics are actually better because the model now learns real patterns.

3. Business Applications

This model could serve as a project greenlight decision. Publishers receive hundreds of game pitches annually but can only fund 10-20 projects, making project selection critical. This model serves as a quantitative decision support tool by predicting hit probability based on proposed game specifications (platform, genre, publisher track record). By inputting game concepts before development begins, publishers can rank projects by success likelihood and fund only those exceeding a defined threshold (e.g., >70% hit probability). This data-driven approach prevents costly failures, avoiding just one $5M failed game development pays for years of data science infrastructure investment.

Another application would be for Regional distribution strategies. Launching globally requires a lot of funding, but with a model like this one, it would help game businesses pinpoint what countries to push for more game releases in said countries for physical game sales. Enabling focused launches that build momentum before global expansion. For instance, an RPG showing high Japanese sales concentration in similar historical titles might launch in Japan first, build a fanbase, then expand westward six months later, saving $2M in upfront localization while maximizing regional impact. This focused approach reduces costs 40-60% while improving overall success rates.

4. Learning Reflections

Before this lab, I understood data leakage conceptually but had never experienced it firsthand. Discovering it in my own model was fun in its own way. The moment I wrote out the math, Total_Regional_Sales = NA + EU + JP + Other = Global_Sales , I realized the model had

direct access to the answer. This moment changed how I approach not just my assignment, but feature engineering too. Working with only 12% positive samples taught me why F1-score, precision, recall, and ROC AUC matter more than accuracy for imbalanced datasets. Initially, seeing accuracy drop from 99% to 91% felt like I did something wrong, but cross checking stuff, I came to accept that anyone can do and overlook mistakes, so it's best to cross check and make sense of the data we are working with. I learned that machine learning success is measured by insight and practice, not just performance numbers.

5. Limitations and Improvements

Current Limitations:

1. **Missing Business Context:** No data on marketing budgets, review scores, franchise value, or development team track records
2. **Temporal Oversimplification:** Model treats all years equally despite dramatic market evolution (mobile gaming, digital distribution, game-as-service models)
3. **Arbitrary Success Definition:** The 1 million unit threshold ignores development costs, platform expectations (indie vs. AAA), and different business models
4. **Geographic Aggregation:** Broad regional data misses cultural factors and important emerging markets

Proposed Improvements:

- **Enhanced features:** Metacritic scores, marketing spend, IP/franchise indicators, competition indices
- **Time-series validation:** Train on 1980-2010, test on 2011-2020 to simulate real prediction scenarios
- **Ensemble methods:** Combine Decision Trees with Random Forest and Gradient Boosting
- **Business-oriented thresholds:** Profit-based definitions using estimated development costs rather than arbitrary unit counts

6. Connections to Theory: Bridging Classroom and Practice

This lab really helped me understand the basics of supervised learning from the PowerPoints and my own research, like needing a clear target variable, making sure your features don't depend on the target, which I messed up at first, and separating your training and testing data to get honest results. When I fixed my model from the broken version to the working one, I could actually see the bias-variance tradeoff in action, basically finding the sweet spot where your model isn't too simple but also isn't memorizing the training data. The evaluation metrics finally made sense to me in a practical way instead of just being formulas. Precision tells you "when I predict a game will be a hit, how often am I right?" which matters because you don't want to waste money on games that won't succeed. Recall answers "out of all the actual hit games, how many did I catch?" which is important so you don't miss good opportunities. ROC AUC measures how well the model ranks games from most to least likely to be hits. The biggest lesson I learned was "garbage in, garbage out", even the fanciest algorithm can't fix bad data or

features that leak information, so you have to check your data integrity before building models because having a perfect algorithm doesn't matter if it's learning the wrong patterns.

7. Conclusion

This lab taught me way more than just how to use Decision Trees, it helped change how I think about data and what makes a model actually good. Going from 99% accuracy that was a false positive, to 91% accuracy, it showed me that I still have a lot to learn about data science. Before this lab, I was all about getting the code in and code out, but now I understand that understanding what's actually happening is more important than just chasing good metrics. I learned to be a bit more suspicious when results look too perfect and to really examine my features to make sure they're not accidentally giving away the answer. The toughest part of machine learning isn't coding or building the model, it's making sure the model is learning real patterns and not just cheating. Honestly, I learned way more from messing up and fixing my broken model than I would have if everything had worked perfectly the first time. Now I know that perfect metrics are actually a warning sign that something's probably wrong, not something to celebrate. This whole experience showed me that you can know all the technical stuff but still be totally wrong if you're not thinking critically about what your model is actually doing.